

GRAVITAS COMPILER - GI3I

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

14 de junio de 2026

Nombres	Apellido	Legajo	E-mail
Antonella Belén	Cragno	65542	acragno@itba.edu.ar
María Eugenia	Arrinda	65580	marrinda@itba.edu.ar
Rocco	Perrone	65628	rperrone@itba.edu.ar

1. Introducción	2
2. Modelo Computacional	2
a. Dominio	2
b. Lenguaje	3
3. Implementación	3
a. Frontend	3
b. Backend	5
c. Adicionales	7
d. Dificultades Encontradas	7
4. Futuras Extensiones	8
5. Conclusiones	8
6. Referencias	8
7. Bibliografía	8

1. Introducción

Este informe documenta el diseño y desarrollo de **Gravitas**, un lenguaje específico de dominio para describir sistemas físicos bidimensionales y generar sus diagramas de cuerpo libre.

El lenguaje permite declarar cuerpos, superficies, fuerzas, sistemas de referencia y distancias mediante una sintaxis declarativa. El compilador valida la descripción ingresada y genera como salida un archivo LaTeX con código TikZ, evitando que el usuario deba definir manualmente coordenadas o instrucciones gráficas.

Gravitas es un lenguaje descriptivo: representa el sistema indicado, pero no resuelve ecuaciones ni simula su movimiento. El compilador fue desarrollado en C utilizando Flex y Bison sobre el proyecto base provisto por la cátedra. El código fuente y la documentación del proyecto se encuentran disponibles en: <https://github.com/mariu23/GravitasCompiler>.

2. Modelo Computacional

El modelo de Gravitas representa sistemas mecánicos planos. Cada programa contiene uno o más sistemas independientes; cada sistema agrupa los cuerpos, superficies, fuerzas y relaciones que deben aparecer en un mismo diagrama. El proceso de compilación parte del código fuente, construye un AST, aplica validaciones semánticas y genera el código del gráfico. Para obtener la representación visual final, el usuario compila ese archivo con una distribución de LaTeX compatible con TikZ.

a. Dominio

El dominio de Gravitas son los sistemas mecánicos planos que aparecen típicamente en los problemas de estática y dinámica elemental: uno o más cuerpos rígidos, eventualmente apoyados sobre superficies, sometidos a un conjunto de fuerzas. La unidad de organización es el sistema (*system*), que agrupa todos los elementos de un mismo diagrama. Un programa puede contener varios sistemas independientes, cada uno con su propio identificador y su propio diagrama de salida.

Dentro de un sistema se distinguen las siguientes entidades:

- **Cuerpos (*body*).** Cada cuerpo tiene un identificador único dentro del sistema y, opcionalmente, una forma (*block* o *sphere*; por defecto, *block*), una masa con su unidad, un conjunto de fuerzas y un coeficiente de fricción propio. Un cuerpo puede declararse apoyado sobre otro (*body B on A*), lo que modela apilamientos. El lenguaje admite cadenas de apilamiento y prohíbe los ciclos.
- **Superficie (*surface*).** Una superficie puede ser horizontal (*horizontal*), un plano inclinado (*incline*) con un ángulo, o un polígono (*polygon*) definido por una secuencia de vértices. Toda superficie puede tener fricción, caracterizada por un coeficiente estático y uno cinético.
- **Fuerzas.** Se distinguen dos clases: las fuerzas explícitas y las implícitas. Las primeras (*force*) tienen nombre, una magnitud con unidad y una dirección, que puede expresarse como un ángulo absoluto o como paralela a la superficie. Las implícitas (*implicit*) son las que el dominio

físico introduce de manera estándar: peso, normal y fricción, y pueden recibir un nombre propio (p. ej. *weight as W*).

- Sistema de referencia (*reference frame*). Cada sistema puede definir un sistema de ejes, ya sea absoluto o alineado con la superficie, anclado a un cuerpo determinado.
- Distancias (*distance*). Una distancia relaciona dos cuerpos y puede expresarse en forma polar (magnitud y ángulo) o cartesiana (componentes x e y).
- Gravedad (*gravity*) y unidades por defecto (*units*) a nivel de sistema.

Para acotar el alcance del proyecto se tomaron decisiones de diseño que delimitan el dominio. El lenguaje es estrictamente descriptivo y bidimensional: no calcula resultantes, reacciones, ni incógnitas, y no modela la evolución temporal del sistema. El objetivo es la representación gráfica de un enunciado de cuerpo libre, no su resolución.

b. Lenguaje

Gravitas es un lenguaje declarativo y orientado a bloques: el usuario no indica cómo dibujar cada elemento sino que declara su existencia y sus propiedades, y delega en el compilador el cálculo de posiciones y el trazado. La estructura raíz es uno o más bloques `system Id { ... }`. Dentro de un sistema, los elementos se declaran mediante bloques anidados delimitados por llaves. Las propiedades simples terminan en punto y coma y, en general, el orden de los elementos dentro de un sistema o de un cuerpo es libre. La única exigencia estructural es que el sistema contenga al menos un cuerpo.

Los valores numéricos admiten enteros y reales, notación exponencial ($1.5e-3$), las constantes π y e , y expresiones aritméticas con multiplicación, división, negación y paréntesis, como $\pi/4$ y $2*\pi$. Las magnitudes pueden acompañarse de unidades: masa (kg , g , mg), fuerza (N , kN), distancia (m , cm , mm , km) y ángulo (deg o el símbolo $^\circ$, y rad). Los comentarios pueden ser de línea (`//`) o de bloque (`/* ... */`).

3. Implementación

El compilador se organiza en frontend y backend sobre el proyecto base^[1]. El frontend transforma el programa de entrada en un stream de tokens (Flex) y construye un Árbol de Sintaxis Abstracta o AST (Bison); el backend recorre el AST para verificar la consistencia semántica y, finalmente, genera el documento LaTeX/TikZ.

a. Frontend

El **analizador léxico** reconoce las siguientes categorías de tokens: las palabras clave del dominio (`system`, `body`, `surface`, `force`, `implicit`, `reference`, `frame`, `distance`, `gravity`, `units`, `mass`, `magnitude`, `direction`, `angle`, `friction`, `static`, `kinetic`, `type`, `block`, `sphere`, `polygon`, `horizontal`, `incline`, `weight`, `normal`, `parallel`, `to`, `on`, `aligned`, `with`, `absolute`, `as`); las unidades; las constantes π y e con acciones dedicadas que almacenan su valor numérico y su texto fuente; los números, con soporte para notación exponencial; los indicadores; y los símbolos (llaves, paréntesis, coma, punto y coma, guion, asterisco y barra). Los comentarios de bloque se manejan mediante una condición de arranque (*start condition*) propia, y el scanner se configura como reentrante.

Para el **análisis sintáctico**, se definió la gramática G , la cual determina los símbolos no terminales y las producciones libres de contexto que generan el lenguaje. El parser es de tipo push y reentrante, con una unión de valores semánticos, precedencia para los operadores $*$ y $/$, y destructores asociados a cada tipo de no terminal para liberar la memoria del AST ante un error de análisis. Un rasgo de diseño es la construcción incremental por fusión del AST: cada ítem de un sistema (o de un cuerpo) produce un objeto parcial, y producciones de merge los combina en un único nodo. Esto permite declarar los elementos en cualquier orden sin multiplicar las producciones. El **Cód. (3.1)** presenta un subconjunto representativo de la gramática.

Otra restricción se expresa directamente en la forma de la gramática: un sistema sin cuerpos no es derivable, de modo que un programa como `system A { gravity 9.8; }` se rechaza ya en el análisis sintáctico.

```

program      -> systemList
systemList   -> system | systemList system
system       -> "system" ID "{" systemItemsWithBody "}"

bodyDeclaration -> "body" ID ";"
                | "body" ID "{" bodyItems "}"
                | "body" ID "on" ID ";"
                | "body" ID "on" ID "{" bodyItems "}"

surfaceDeclaration
  -> "surface" "{" "type" "horizontal" ";" optFriction "}"
    | "surface" "{" "type" "incline" ";" "angle" angle ";" optFriction "}"
    | "surface" "{" "type" "polygon" ";" vertices ";" optFriction "}"

forceDeclaration -> "force" ID "{" magnitude direction "}"

implicitForce  -> "weight" | "normal" | "friction"
                | "weight" "as" ID
                | "normal" "as" ID
                | "friction" "as" ID

```

Código 3.1: Subconjunto representativo de la gramática.

El **árbol AST** se compone de nodos para *Program*, *System*, *Units*, *Surface*, *Friction*, *Mass*, *Direction*, *Force*, *ImplicitForce(List)*, *Body*, *ReferenceFrame*, *Point* y *Distance*, más un tipo *Value* que guarda el valor numérico junto con su texto fuente, para poder reproducirlo luego en LaTeX. El **Cód. (3.2)** muestra dos de estos nodos.

```

typedef struct Body {
    char      *name;
    char      *parentBodyName; /* "body B on A" -> "A" */
    BodyShape shape;           /* default | block | sphere */
    Mass      *mass;
    AstList   *forces;          /* fuerzas explícitas */
    ImplicitForceList *implicitForces;
    Friction  *friction;
} Body;

typedef struct Value {
    double numericValue;
    char   *sourceText; /* p.ej. "PI/4" -> \frac{\pi}{4} en la salida */
} Value;

```

Código 3.2: Dos nodos del AST (versión simplificada).

b. Backend

En el **análisis semántico**, se recorre el AST sistema por sistema. Para cada uno se construye una tabla con los nombres de los cuerpos y se aplican las validaciones propias del dominio. Concretamente, se verifica que:

1. El sistema tiene al menos un cuerpo.
2. Los nombres de los cuerpos son únicos.
3. Todo cuerpo apoyado sobre otro referencia a un cuerpo existente.
4. No existen ciclos de apoyo entre cuerpos.
5. Las superficies de tipo polígono son definidas por al menos tres vértices.
6. Toda distancia se declara entre cuerpos existentes.
7. El sistema de referencia se ancla a un cuerpo existente.
8. Los coeficientes de fricción (tanto de una superficie, como de un cuerpo) no son negativos.
9. Los nombres de las fuerzas son únicos dentro de cada cuerpo.
10. Si se define una dirección paralela, entonces existe por lo menos una superficie en el sistema.
11. Si se define una fuerza implícita normal, entonces existe por lo menos una superficie en el sistema.
12. Si se define una fuerza implícita de fricción, entonces el cuerpo tiene asignado coeficientes de fricción o existe por lo menos una superficie con fricción en el sistema.
13. La masa de un cuerpo es positiva.
14. El valor de la gravedad del sistema no es negativo.
15. Las magnitudes de las fuerzas son positivas.
16. Las distancias polares no son negativas.

La primera se realiza de forma defensiva aunque ese sistema ya sería rechazado la etapa de análisis sintáctico. Las tres últimas son restricciones sensibles al contexto: una fuerza implícita sólo es admisible si el sistema contiene los elementos físicos que la justifican. A diferencia de las restricciones puramente sintácticas, estas no derivan de una limitación de la herramienta de salida, sino de la coherencia física del enunciado.

Por último, se trabajó en la **generación de código**. El destino es un documento LaTeX autónomo que incluye los paquetes necesarios y un bloque *tikzpicture* por cada sistema. El generador implementa un motor de layout sencillo que asigna coordenadas a cada entidad:

- **Posiciones.** Los cuerpos sin apoyo se distribuyen en una horizontal equiespaciada. Un cuerpo apoyado sobre otro se ubica por encima de su soporte (apilamiento vertical).

- **Superficies.** La horizontal se traza como un segmento. El plano inclinado se traza como un segmento rotado según su ángulo, con su rótulo. El polígono se traza recorriendo sus vértices y cerrando el contorno.
- **Cuerpos.** La esfera se dibuja como un círculo y el bloque como un rectángulo, con la masa rotulada debajo.
- **Fuerzas explícitas.** Una flecha de longitud fija en la dirección resuelta: el ángulo absoluto indicado o, si la dirección es paralela a la superficie, el ángulo de la primera superficie.
- **Fuerzas implícitas.** El peso como flecha hacia abajo (W), la normal hacia arriba (N) y la fricción hacia la derecha (F_f), con sus nombres opcionales.
- **Distancias.** Una flecha de doble punta entre los centros de los cuerpos, rotulada con la magnitud (polar) o las componentes (cartesiana).
- **Sistema de referencia.** Ejes x e y en color, rotados según la superficie si se eligió alineado con la superficie.

El **Cód. (3.3)** es un programa de entrada; el **Cód. (3.4)** es el bloque tikzpicture que produce el generador (precedido en la salida real por un preámbulo standalone con los paquetes); y la **Fig. (3.1)** muestra el diagrama resultante al compilar esa salida.

```
system A {
  surface {
    type horizontal;
    friction {
      static 0.4;
      kinetic 0.3;
    }
  }

  body B {
    implicit weight, normal, friction;
  }
}
```

Código 3.3: Programa de entrada

```
\documentclass[border=5pt]{standalone}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
\usepackage{amsmath}
\usepackage{tikz}
\usetikzlibrary{arrows,calc}
\begin{document}
  \begin{tikzpicture}[scale=0.8,>=stealth]
    \draw[thick] (-2.100000, -1.500000) -- node[pos=0.68,sloped,below=1pt,inner
sep=1pt] {\scriptsize  $F_s = 0.4, \text{quad } F_k = 0.3$ } (2.100000, -1.500000);
    \begin{scope}[shift={(0.000000,-0.970000)},rotate=0.000000,transform shape]
      \draw[thick,fill=gray!20] (-0.700000, -0.450000) rectangle (0.700000,
0.450000);
      \node at (0, 0) {\mathcal{B}};
    \end{scope}
    \draw[->,thick] (0.000000, -1.460000) -- (0.000000, -3.660000);
    \node[anchor=north] at (0.000000, -3.840000) {\mathcal{W}_{\mathcal{B}}};
    \draw[->,thick] (0.000000, -0.480000) -- (0.000000, 1.720000);
    \node[anchor=south] at (0.000000, 1.900000) {\mathcal{N}_{\mathcal{B}}};
```

```

\draw[->,thick] (0.740000, -0.970000) -- (2.940000, -0.970000);
\node[anchor=west] at (3.120000, -0.970000) {$F_{f,B}$};
\node[anchor=south] (system-title) at (0.000000, 3.280000) {\textbf{System:
A}};
\end{tikzpicture}
\end{document}

```

Código 3.4: Bloque TikZ generado para el programa anterior.

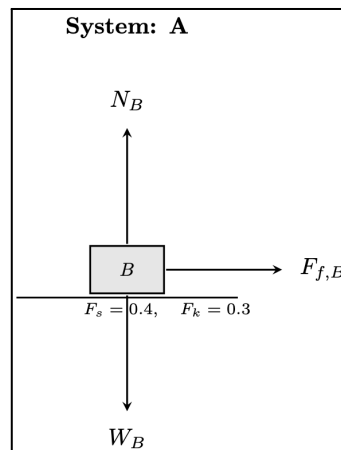


Figura 3.1: Diagrama resultante al compilar el bloque TikZ anterior.

c. Adicionales

Más allá del núcleo mínimo, el lenguaje incorpora algunas prestaciones adicionales:

- Constantes π , e y un subconjunto de expresiones aritméticas ($*$, $/$, $-$ y $()$), con preservación del texto fuente para producir una salida matemática legible.
- Nombres propios para las fuerzas implícitas (*weight as W*).
- Apilamiento de cuerpos (*body B on A*) con detección de ciclos.
- Superficies poligonales de vértices arbitrarios, con unidades por coordenada.
- Múltiples sistemas por programa, cada uno con su propio diagrama.
- Un conjunto de validaciones sensibles al contexto que aseguran la coherencia física del enunciado.

d. Dificultades Encontradas

Fuera de las dificultades técnicas habituales en este tipo de proyecto - principalmente la cohesión y adaptación al código base provisto - las dificultades más relevantes fueron las siguientes:

- **Diseño y acotamiento del dominio.** El principal desafío fue delimitar el alcance. Un dominio físico es muy amplio y hubo que decidir qué representar (estática 2D, sin resolución) para que el proyecto fuera abordable sin volverse trivial ni inabarcable.

- **Layout estático.** El motor de posicionamiento es heurístico: los cuerpos se ubican en una fila o apilados con separaciones fijas, sin considerar la geometría de las superficies. Por eso un cuerpo puede quedar flotando sobre la superficie en lugar de apoyado, y en sistemas complejos pueden solaparse flechas y rótulos. Además, el rótulo del sistema se ubica a una altura fija, lo que deja espacio vertical sobrante en diagramas pequeños.

4. Futuras Extensiones

Si bien el lenguaje es funcional dentro de su alcance, el desarrollo evidenció oportunidades claras de mejora:

- **Layout dinámico.** Calcular un bounding box a partir del contenido para dimensionar el lienzo y ubicar el rótulo, y apoyar efectivamente los cuerpos sobre las superficies según su geometría.
- **Unidades por defecto efectivas.** Aplicar las unidades del bloque units a las magnitudes que las omiten.
- **Flechas proporcionales.** Escalar la longitud de las flechas de fuerza según su magnitud.
- **Resolución del DCL (mayor alcance).** A partir del sistema descrito, calcular la fuerza resultante o las incógnitas, pasando de un lenguaje puramente descriptivo a uno que también resuelva problemas de física.
- **Nuevos formatos y formas.** Salida directa a SVG/PNG y un catálogo más amplio de formas de cuerpo.
- **Incorporar cuerdas, poleas, resortes y otras relaciones mecánicas,** como un *body inside body* y más opciones para ubicar los cuerpos sobre superficies.

5. Conclusiones

La realización de este proyecto permitió llevar los conceptos de compiladores a una implementación funcional de extremo a extremo: un programa escrito en Gravitas se tokeniza, se parsea en un AST, se valida contra las reglas del dominio y se traduce a un diagrama de cuerpo libre en LaTeX/TikZ. El lenguaje cumplió con su objetivo principal: abstraer el trazado manual y permitir describir un sistema físico de manera simple y legible.

Fue una experiencia muy didáctica ver el ciclo de compilación completo en acción y comprobar cómo una descripción declarativa se transforma en un resultado visual. La solución resulta sólida dentro de su alcance y sienta una base extensible para futuras versiones.

6. Referencias

- [1] Cátedra de Autómatas, Teoría de Lenguajes y Compiladores. Flex-Bison-Compiler. Disponible en: <https://github.com/agustin-golmar/Flex-Bison-Compiler> (branch production, tag v2.0.0).

7. Bibliografía

- Material provisto por la cátedra de Autómatas, Teoría de Lenguajes y Compiladores.
- T. Tantau. The TikZ and PGF Packages Manual. CTAN, The Comprehensive TeX Archive Network. Disponible en: <https://ctan.org/pkg/pgf>.